

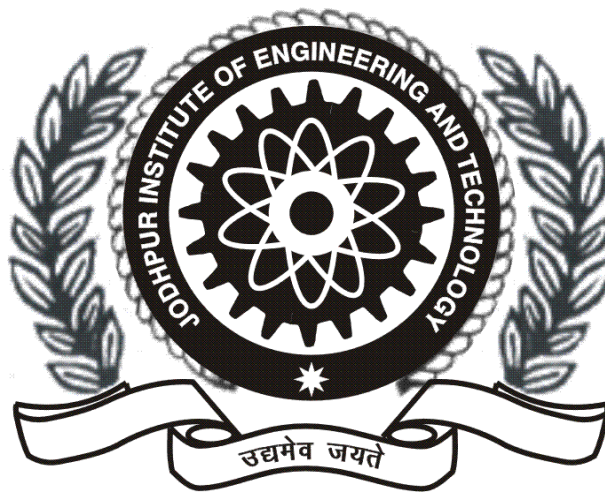
Software Requirements Specification

(SRS)

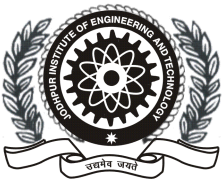
For

BusMate

20th September 2025



Department of Computer Science Engineering (AIML/DS/CYS)
Jodhpur Institute of Engineering and Technology
Session 2025-26



APPROVAL and Compulsory Review LIST		
Role	Name	Date
Lab Faculty/ Supervisor	Ms. Khushboo Parashar	20 th September 2025

Version Tracking			
Date	Version	Comments	Author
20 th September 2025	1.0	Completed Initial Draft	Sneha Dadhich Noorun Fatima Khan

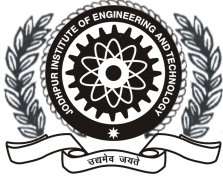


Table of Contents

1.0	INTRODUCTION.....	3
1.1	Scope.....	3
1.2	Technologies to be used.....	3
2.0	GENERAL REQUIREMENTS.....	6
2.1	Functionalities.....	6
2.2	Use Case Model Diagrams.....	6
2.3	General Constraints.....	7
2.4	Supplementary requirements.....	8
3.0	DEFINITION, ACRONYMS, AND ABBREVIATIONS.....	8
4.0	REFERENCES.....	9



1.0 INTRODUCTION

1.1 Scope

The BusMate system is designed to address the specific needs of educational institutions and has the following scope:

1. **Institution-Specific Application:** Customized for colleges, schools, or universities where buses are the primary mode of student transport.
2. **Dynamic Bus Allocation:** Capable of handling daily changes in bus assignments due to driver availability or maintenance issues.
3. **Student-Focused Interface:** Provides a simple, mobile-friendly interface for students to access bus information and track locations in real time.
4. **Admin-Focused Interface:** Facilitates bus management, route planning, driver assignment, and transport fee monitoring.
5. **Scalability:** Can be expanded to include multiple campuses, additional transport modes, and integration with smart campus systems.
6. **Security and Compliance:** Ensures secure access and fee compliance through QR-code scanning and optional AI-driven monitoring features.

1.2 Technologies to be used

Frontend

- HTML / CSS → Basic structure and styling of the web pages.
- Jinja2 Templates → Dynamically render student/admin dashboards with Python backend data.
- JavaScript / AJAX → Interactive elements, form validations, and real-time updates (e.g., bus status, geolocation tracking).



Backend

- Python → Core programming language.
- Flask / FastAPI → Web framework to serve Jinja templates and handle API endpoints.
- Werkzeug → For secure request handling and authentication.

Database & Storage

- SQLite / MySQL / PostgreSQL → Store user data, bus schedules, geolocation logs, QR codes.
- SQLAlchemy → ORM for easy database interactions.
- Pandas → Optional for data handling and reporting.

Authentication & Security

- Flask-Login / Flask-Security → Manage student/admin login sessions.
- bcrypt / hashlib → Secure password hashing.
- JWT Tokens (optional) → For API authentication.

QR Code Verification

- qrcode → Generate QR codes for student tickets or bus passes.
- pyzbar → Scan and decode QR codes using images or webcam.
- OpenCV → Capture QR codes from webcam or uploaded images.

Geolocation & Mapping

- geopy → Convert addresses to coordinates and vice versa.
- geocoder → Get current user location via IP or GPS.
- Google Maps API / OpenStreetMap API → Fetch routes, calculate distance, or display maps.
- folium / leaflet.js → Display bus locations and routes interactively on a map.
- pynmea2 / GPS modules → Read GPS device data if bus hardware is used.

Software Requirements Specification



Other Tools

- Requests → For making API calls (e.g., Google Maps API).
- Base64 → Encode/decode images or QR codes.
- Docker (optional) → Containerize the application for deployment.



2.0 GENERAL REQUIREMENTS

2.1 Functionalities

Student Module

- View assigned bus details (bus number, route, timing).
- Check real-time bus location on map (geolocation feature).
- Scan and verify QR codes for boarding verification.
- Receive notifications about delays or changes in schedule.
- Submit feedback or complaints regarding bus service.

Admin Module

- Manage bus schedules, routes, and stops.
- Assign buses to students or groups.
- Track buses in real-time using geolocation data.
- Generate reports on bus occupancy, student usage, and feedback.
- Validate QR codes scanned by students for verification.

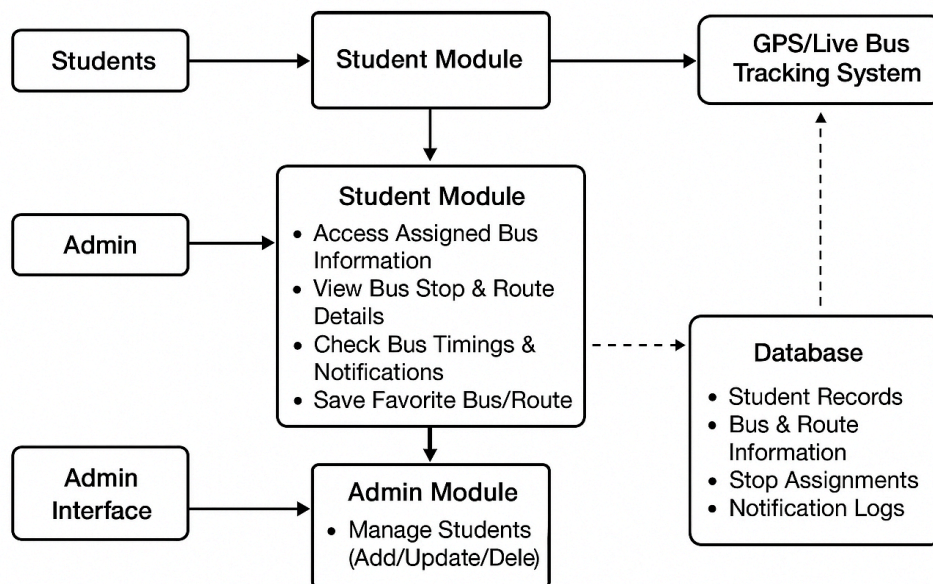
System Features

- Secure login and authentication for students and admins.
- Data storage and retrieval for schedules, QR codes, and geolocation.
- API integration with Google Maps / OpenStreetMap for route mapping.
- Dashboard visualizations for bus tracking and reporting.



2.2 Use Case Model Diagrams

System Functional Block Diagram



2.3 General Constraints

- The system must be accessible on desktop and mobile browsers.
- Only authorized users (students/admins) can access respective modules.
- QR verification must work under low-light or low-resolution conditions.
- Geolocation tracking must be accurate within ± 50 meters.
- The system should handle concurrent access by multiple students and buses without performance degradation.
- Database operations must ensure data consistency and avoid loss.



2.4 Supplementary requirements

- Performance: Real-time tracking updates should have minimal latency (< 5 seconds).
- Reliability: System should be available 99% of the time with backup mechanisms for server downtime.
- Usability: User-friendly dashboards and easy navigation for both students and admins.
- Security:
 - Passwords must be hashed and stored securely.
 - QR codes should have time-based or unique identifiers to prevent misuse.
 - API keys for maps or GPS services must be securely stored.
- Maintainability: System should allow easy updates of bus schedules, routes, and modules.
- Portability: Application should run on any OS that supports Python, Flask/FastAPI, and required libraries.

3.0 Definition, Acronyms, and Abbreviations

Acronym	Definition
GPS	Global Positioning System – used for tracking bus locations.
QR	Quick Response – a type of barcode used for student bus verification.
API	Application Programming Interface – allows communication between BusMate and external services like Google Maps.
DBMS	Database Management System – manages storage and retrieval of system data.
HTML	HyperText Markup Language – used to structure frontend pages.



CSS	Cascading Style Sheets – used to style frontend pages.
JS	JavaScript – adds interactivity to web pages.
UML	Unified Modeling Language – used for system diagrams like use case models.

4.0 References

1. Flask Documentation – <https://flask.palletsprojects.com/>
2. MySQL Documentation – <https://dev.mysql.com/doc/>
3. Google Maps API – <https://developers.google.com/maps>